

IITB Triples

Team SAKEC

In standard Unitary Coupled Cluster (UCC) theory:

Singles (T1): Moving 1 electron.

Doubles (T2): Moving 2 electrons.

Triples (T3): Moving 3 electrons simultaneously.

A true Triple excitation (T3) is a distinct, highly complex operator that moves 3 electrons all at once.

How the IITB Code Creates "Pseudo-Triples"

True T3 operators create massive quantum circuits that are too deep for current quantum computers. The IITB patch uses a brilliant algorithmic shortcut to simulate Triples using *only* Doubles.

Here is exactly what is happening in the coupling phase of the script:

1. **Grabbing Two Doubles:** The script takes a highly impactful Double excitation from your pruned list and pairs it with a custom **Sh** or **Sp** operator. These **Sh** and **Sp** operators are also formatted mathematically as Double excitations (moving 2 electrons).
2. **Forcing an Overlap:** Look at this specific line of code during the pairing process: `if i == d or j == d:`. This line acts as a strict filter. It ensures that the two Double operators are only paired together if they share **exactly one orbital index**.
3. **The "Triple" Effect:** If Operator A moves electrons from orbitals 1, 2 -> 3, 4, and Operator B moves electrons from 4, 5 -> 6, 7, they overlap at orbital 4. When the quantum circuit runs these two operators sequentially, their interaction effectively moves 3 electrons (1, 2, 5 -> 3, 6, 7).

So, the IITB code cleverly combines a **Double + an Overlapping Double** to synthesize the physical effect of a Triple excitation without the massive computational cost!

Flow of the Code Patch

Phase 1: Pruning Doubles

The flow begins by aggressively filtering out useless double excitations to save quantum resources.

- The script initializes a standard Unitary Coupled Cluster (UCC) ansatz restricted entirely to double ('d') excitations.
- It maps these fermionic double excitations into Pauli strings so the quantum computer can process them.
- A loop isolates every single double excitation and builds a miniature quantum circuit containing only that one operator.
- A fast, single-parameter VQE is executed for each isolated circuit to evaluate its independent energy contribution.
- The script calculates the difference between the starting reference energy and the new minimized energy.
- If this specific double excitation drops the energy by more than a threshold of $1e-5$ Hartree, its operator and optimal parameter are saved.
- If the energy drop is less than the threshold, the excitation is permanently discarded.
- Finally, the surviving double excitations are mathematically sorted in descending order based on exactly how much they lowered the energy.

Flow of the Code Patch

Phase 2: Introducing Custom Excitations

Next, the script prepares specialized sets of spatial orbital configurations.

- Two hardcoded lists of orbital pairs are introduced: the **Sh_list** (hole-related spatial orbitals) and the **Sp_list** (particle-related spatial orbitals).
- Custom UCC objects are created to mathematically represent these specific spatial pairs.
- These custom operators are subsequently mapped into Pauli strings, generating **sh_pauli** and **sp_pauli** lists.

Phase 3: Entanglement (Coupling)

Instead of adding the **Sh** and **Sp** operators blindly, the script tests them to see if they geometrically correlate with the surviving double excitations.

- The code loops through the ranked list of successful double excitations.
- For every successful double excitation, it analyzes the specific orbitals involved (labeled as **i, j** moving to **a, b**).
- It systematically scans the **Sh_list** to check if a custom operator shares an occupied orbital (checking if **i == d** or **j == d**) with the current double excitation.

Flow of the Code Patch

Phase 3: Continue

- If a geometric match is found, it creates a new two-parameter test circuit combining the successful double excitation and the matching **Sh** operator.
- A secondary mini-VQE is executed on this paired circuit, utilizing the previously found optimal parameter to accelerate convergence.
- If this paired circuit lowers the energy beyond a **1e-5** Hartree threshold relative to the double excitation alone, the **Sh** operator and its parameter are officially appended to the final circuit lists.
- The exact same orbital-matching and VQE-testing process is repeated for the **Sp_list**, but checking for shared virtual/unoccupied orbitals (**a == q** or **b == q**).

Flow of the Code Patch

Phase 4: Appending Singles and Final Assembly

- A standard UCC ansatz generates all possible single ('s') excitations.
- Every single excitation is automatically appended to the final list of operators without undergoing any pruning or testing.
- The final quantum circuit (`ansatz = EvolvedOperatorAnsatz(...)`) is assembled using the aggregated list of Pauli strings: the pruned doubles, the successfully paired Sh/Sp operators, and all singles.
- An initial parameter array (`padded_initial_point`) is constructed by combining the pre-optimized parameters from the pruning tests with zeros for the newly added single operators.

Phase 5: Execution

- The fully customized ansatz, along with the heavily pre-optimized `padded_initial_point`, is passed into the main VQE solver (`vqe_solver = VQE(...)`).
- Because the ansatz has been stripped of useless operators (reducing circuit depth) and the solver is handed an excellent starting point, the main VQE can converge on the ground state energy drastically faster than a standard simulation.

Understanding with code

The script initializes a standard Unitary Coupled Cluster (UCC) ansatz restricted entirely to double ('d') excitations.

```
var_form = UCC(  
    num_spatial_orbitals=active_num_spatial_orbitals,  
    num_particles=num_particles,  
    qubit_mapper=mapper,  
    initial_state=initial_state,  
    excitations='d'  
)
```

It maps these fermionic double excitations into Pauli strings so the quantum computer can process them.

```
fer_excitation_op = var_form.excitation_ops()  
excitation_list_pauli = list()  
for ex in fer_excitation_op:  
    excitation_list_pauli.append(mapper.map(ex))
```

Understanding with code

A loop isolates every single double excitation and builds a miniature quantum circuit containing only that one operator.

```
for i in range(len(excitation_list_pauli)):
    var_form1 = EvolvedOperatorAnsatz(excitation_list_pauli[i],
    initial_state=initial_state)
```

If this specific double excitation drops the energy by more than a threshold of $1e-5$ Hartree, its operator and optimal parameter are saved.

```
if abs(initial_energy - E1) > 1e-5:
    energy_list.append(E1)
    difference_E.append(abs(initial_energy-E1))
    optimal_list.append(op_pt[0])
```

Finally, the surviving double excitations are mathematically sorted in descending order based on exactly how much they lowered the energy.

```
difference_E1=difference_E.copy()
difference_E1.sort(reverse=True)
```


Understanding with code

Two hardcoded lists of orbital pairs are introduced: the Sh_list and the Sp_list.

```
Sh_list= [(( 6 , 0 ), ( 9 , 2 )), (( 6 , 1 ), ( 9 , 2 )), ...]  
Sp_list= [(( 6 , 3 ), ( 9 , 4 )), (( 6 , 3 ), ( 9 , 5 )), ...]
```

Custom UCC objects are created to mathematically represent these specific spatial pairs mapped into Pauli strings, generating sh_pauli and sp_pauli lists.

```
ucc_sh = UCC(..., excitations = S_list_h, ...)  
sh_fer_ops = ucc_sh.excitation_ops()  
sh_pauli = list()  
for fer in sh_fer_ops:  
    sh_pauli.append mapper.map(fer)  
#(Note: This exact block is duplicated for `ucc_sp` and `sp_pauli`)
```

The code loops through the ranked list of successful double excitations, it analyzes the specific orbitals involved (labeled as i, j moving to a, b).

```
for index in range(len(dob_excitation_list)):  
    double = dob_excitation_list[index]  
    i, j, a, b = double[0][0], double[0][1], double[1][0], double[1][1]
```

Understanding with code

It systematically scans the `Sh_list` to check if a custom operator shares an occupied orbital (checking if `i == d` or `j == d`) with the current double excitation.

```
for kkk in range(len(Sh_list)):
    sh = Sh_list[kkk]
    p, q, c, d = sh[0][0], sh[0][1], sh[1][0], sh[1][1]
    if i == d or j == d:
```

If a geometric match is found, it creates a new two-parameter test circuit combining the successful double excitation and the matching `Sh` operator.

```
dummy_pauli = list()
dummy_pauli.append(final_dob_list[index])
dummy_pauli.append(sh_pauli[kkk])
ucc_custom_2 = EvolvedOperatorAnsatz(dummy_pauli, initial_state = initial_state)
```

A secondary mini-VQE is executed on this paired circuit, utilizing the previously found optimal parameter to accelerate convergence.

```
init_par = [dob_params[index]]+[0.01]
vqe1 = VQE(estimator, ucc_custom_2, optimizer=optimizer, initial_point=init_par)
```

Understanding with code

If this paired circuit lowers the energy beyond a $1e-5$ Hartree threshold relative to the double excitation alone, the Sh operator and its parameter are officially appended.

```
if abs(E1-ref_E) > 1e-5:
    final_excitations_list.append(sh)
    final_parameters_list.append(op_pt[-1])
    final_pauli_list.append(sh_pauli[kkk])
```

A standard UCC ansatz generates all possible single ('s') excitations. Every single excitation is automatically appended.

```
var_form_s = UCC(..., excitations='s')
fer_excitation_op_s = var_form_s.excitation_ops()
for ex in fer_excitation_op_s:
    final_pauli_list.append mapper.map(ex))
```

The final quantum circuit is assembled using the aggregated list of Pauli strings. An initial parameter array... is constructed by combining the pre-optimized parameters with zeros for the newly added single operators.

```
ansatz = EvolvedOperatorAnsatz(final_pauli_list, initial_state=initial_state)
padded_initial_point = final_parameters_list + [0.0] * len(fer_excitation_op_s)
```

THANK YOU